Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Develop Multi-Agent AI Applications with AutoGen

TGS Ref No: TGS-2024045806

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 9

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
8.0	1 March 2024	Legacy release — LLM orchestration with LangChain/LangGraph, RAG, and multi-agent frameworks (AutoGen, CrewAI).	Dr. Alfred Ang
9	28 August 2026	Full revamp on Microsoft AutoGen AgentChat 0.7: uv Pre-Lab, 8 AutoGen labs with paired plain-Python and AutoGen scripts, synthetic TechTrend mock data for the tool and RAG labs, retrieval evaluation, and multi-agent teams (RoundRobin, Selector, human-in-the-loop).	Dr. Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Introduction **4**
Course Learning Outcomes **4**
Pre-Lab — Install uv and Prepare the Environment (Lab 0) **4**
Topic 01 — Introduction to LLM AI Orchestration (36%) **5**
Lab 1 — Set Up AutoGen and Run Your First Agent 6
Lab 2 — Configure Model Clients and Structured Output 7
Lab 3 — Equip an Agent with Tools 7
Topic 02 — Retrieval-Augmented Generation (RAG) (36%) **8**
Lab 4 — Build a Knowledge Index with Embeddings and a Vector Database 9
Lab 5 — Build and Evaluate a Grounded RAG Agent 10
Topic 03 — Implementing a Multi-Agent AI Workflow (28%) **10**
Lab 6 — Implement a ReAct Agent with Human-in-the-Loop 11
Lab 7 — Orchestrate a Round-Robin Multi-Agent Team 12
Lab 8 — Route Specialists with SelectorGroupChat (Capstone) 12
Exam Preparation **13**
Glossary **14**

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Introduction

This Learner Guide accompanies the WSQ course Develop Multi-Agent AI Applications with AutoGen (TGS-2024045806), conducted by Tertiary Infotech Academy Pte Ltd. It provides detailed step-by-step instructions for the Pre-Lab and all 8 hands-on labs, organised by the three learning units. You will build multi-agent AI applications with Microsoft AutoGen (AgentChat), grounded in a synthetic company dataset — the TechTrend IT operations handbook and support tickets — so no real or confidential data is ever used.

Every lab folder in the course repository contains a plain Python script (the concept without a framework) and an AutoGen script (the same concept with AutoGen), plus a README. Use this guide alongside the course slides. Keep your API key in labs/.env only — never commit it to Git, and never paste it into prompts, screenshots or answers.

Course Learning Outcomes

- LO1: Evaluate Large Language Model (LLM) AI models by identifying their strengths and limitations.
- LO2: Analyze Retrieval-Augmented Generation (RAG) algorithms to improve efficiency.
- LO3: Assess the feasibility of implementing multi-agent AI applications.

Pre-Lab — Install uv and Prepare the Environment (Lab 0)

Complete this BEFORE Lab 1. All labs run inside one uv-managed Python environment in the labs/ folder of the course repository. uv is a fast, modern Python package and project manager that replaces pip + venv with a single tool.

Step 1 — Install uv

macOS / Linux (Terminal):

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows (PowerShell):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Close and reopen your terminal, then verify the installation:

```
uv --version
```

Step 2 — Get the lab files

```
git clone https://github.com/tertiarycourses/TGS-2024045806-Develop-Multi-Agent-AI-Applications-with-AutoGen.git
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cd TGS-2024045806-Develop-Multi-Agent-AI-Applications-with-AutoGen/labs
```

Step 3 — Install Python and the AutoGen packages

uv sync reads pyproject.toml, installs a private Python 3.10+ interpreter if needed, creates .venv/ and installs AutoGen AgentChat, the OpenAI extension, ChromaDB and everything else the labs use:

```
uv sync
```

Step 4 — Set your OpenAI API key

```
cp .env.example .env

# edit .env so it contains:  OPENAI_API_KEY=sk-...
```

Step 5 — Verify

```
uv run python -c "import autogen_agentchat, autogen_ext, chromadb;
print('AutoGen labs environment OK')"
```

Note: If a later lab reports ModuleNotFoundError, you probably ran plain python — always use the uv run prefix so the .venv environment is used.

Topic 01 — Introduction to LLM AI Orchestration (36%)

LLM strengths & limitations · AI agents · AutoGen setup · model clients · tools

Key concepts

- An LLM generates text token by token from a prompt; it is stateless, has a knowledge cut-off, and can hallucinate — the limitations that orchestration exists to manage.
- Strengths and limitations are evaluated on measurable axes: quality, latency, cost per call, context length, privacy and repeatability (temperature).
- An AI agent wraps an LLM with a role (system message), memory of the conversation, and tools — so it can act, not just answer.
- AutoGen is Microsoft's open-source multi-agent framework: Core (event-driven runtime), AgentChat (task-driven agents & teams), Extensions (model clients, tools) and Studio (no-code UI).
- A model client (OpenAIChatCompletionClient) abstracts the provider — the same agent code runs on OpenAI, Azure, Gemini, or a local Ollama model.
- Function calling turns typed Python functions into tools the model can request; AutoGen's FunctionTool generates the JSON schema from the signature and docstring.
- Structured output (output_content_type with a Pydantic model) converts free text into validated fields that downstream code can branch on.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Async execution (asyncio, run/run_stream, the Console UI) is how AgentChat streams events and scales beyond one blocking call.

Lab 1 — Set Up AutoGen and Run Your First Agent

Objective: LO1 — evaluate an LLM by running it directly, then as an AutoGen agent.

Goal: Verify the uv lab environment from the Pre-Lab, call an LLM directly with the OpenAI SDK, then wrap the same model in an AutoGen AssistantAgent and stream its reply with the Console UI.

What you'll build

A working AutoGen environment plus two runnable scripts: a plain LLM app and your first agent. (Toolchain: uv, Python 3.10+, OpenAI SDK, AutoGen AgentChat, Console UI.)

Step-by-step

1. Complete the Pre-Lab: install uv, then sync the lab environment

```
uv sync
```

2. Create your .env with the course API key (never commit it)

```
cp .env.example .env
```

3. Verify AutoGen imports cleanly in the environment

```
uv run python -c "import autogen_agentchat; print('OK')"
```

4. Run the plain-Python LLM app and note the reply and token count

```
uv run python lab-01-first-autogen-agent/llm_app.py
```

5. Run the same model as an AutoGen AssistantAgent with streaming

```
uv run python lab-01-first-autogen-agent/first_agent.py
```

6. Set temperature to 0 in llm_app.py and re-run twice to compare repeatability

Test it

Both scripts print a model reply with no traceback, and you can state one strength (fluent answers) and one limitation (stateless, non-repeatable at temperature 0.7) you just observed.

Note: The full scripts and a lab README are in labs/lab-01-first-autogen-agent/. Run every command from the labs/ folder with the uv run prefix.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 2 — Configure Model Clients and Structured Output

Objective: LO1 — compare cloud vs local models and constrain output to a schema.

Goal: Run the same prompt against OpenAI's cloud API and a local Ollama model to compare quality, latency and cost, then force an AutoGen agent to reply in a validated Pydantic TicketTriage schema instead of free text.

What you'll build

A model-comparison script with timings, and a triage agent that returns validated JSON fields. (Toolchain: OpenAIChatCompletionClient, Ollama (optional), Pydantic, output_content_type.)

Step-by-step

7. Run the model-client comparison (OpenAI cloud, with timing)

```
uv run python lab-02-model-clients-structured-output/model_clients.py
```

8. Optional: install Ollama and pull a local model, then re-run to compare

```
ollama pull llama3.2
```

9. Read the TicketTriage Pydantic schema — the contract the agent must obey
10. Run the structured-output triage agent on the outage ticket

```
uv run python lab-02-model-clients-structured-output/structured_agent.py
```

11. Change the ticket text to a P4 request and re-run — watch the fields change

Test it

The triage agent returns a valid TicketTriage object with priority P1 for the outage ticket; you can name the trade-offs (latency, cost, privacy, quality) between the cloud and local model.

Note: The full scripts and a lab README are in labs/lab-02-model-clients-structured-output/. Run every command from the labs/ folder with the uv run prefix.

Lab 3 — Equip an Agent with Tools

Objective: LO1 — extend an LLM's limits with function calling and FunctionTool.

Goal: Do raw OpenAI function calling by hand against the synthetic ticket dataset, then let AutoGen's FunctionTool generate the schema, execute the calls and reflect on the results with two read-only ticket tools.

What you'll build

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

A hand-rolled function-calling round trip, and a help-desk agent with lookup_ticket and count_open_tickets tools. (Toolchain: OpenAI function calling, FunctionTool, reflect_on_tool_use, tickets.json mock data.)

Step-by-step

12. Inspect the synthetic ticket data the tools will read

```
cat data/tickets.json
```

13. Run raw function calling: schema → tool_call → execute → final answer

```
uv run python lab-03-agent-tools/function_calling.py
```

14. Read tool_agent.py — one typed Python function becomes one FunctionTool line
15. Run the AutoGen tool agent and watch ToolCallRequestEvent / ToolCallExecutionEvent

```
uv run python lab-03-agent-tools/tool_agent.py
```

16. Ask a question needing BOTH tools and confirm two tool calls are made

Test it

The agent reports the real subject of TT-1005 and the correct count of open P2 tickets — every fact traceable to a tool result, none invented.

Note: The full scripts and a lab README are in labs/lab-03-agent-tools/. Run every command from the labs/ folder with the uv run prefix.

Topic 02 — Retrieval-Augmented Generation (RAG) (36%)

Embeddings · chunking · vector database · similarity search · grounded agents · evaluation

Key concepts

- RAG lets an LLM answer from knowledge outside its training data by retrieving relevant text and stuffing it into the prompt — fresher answers, fewer hallucinations, source citations.
- A text embedding maps text to a vector; semantically similar texts land close together, so meaning becomes measurable as cosine similarity.
- Chunking decides what one retrievable unit is; chunk boundaries (by section, paragraph or tokens, with overlap) directly change retrieval quality.
- A vector database (ChromaDB) stores embeddings and answers approximate nearest-neighbour queries in milliseconds.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- The RAG pipeline: load → chunk → embed → store at indexing time; embed query → similarity search → augment prompt → generate at answer time.
- An agentic RAG agent calls retrieval as a tool, cites source ids, and refuses when evidence is missing — and treats retrieved text as data, never as instructions (prompt-injection defence).
- RAG efficiency is analysed with numbers: hit@k retrieval accuracy on a gold question set, retrieval latency, chunk count and top-k — change one lever, re-measure.

Lab 4 — Build a Knowledge Index with Embeddings and a Vector Database

Objective: LO2 — build the retrieval half of RAG and measure similarity directly.

Goal: Chunk the synthetic TechTrend IT handbook into one-policy chunks, embed each chunk locally, store the vectors in ChromaDB, then run similarity searches and inspect which sources return and at what cosine distance.

What you'll build

A persistent 10-chunk ChromaDB index of the handbook, a similarity-search console, and a first index-backed agent. (Toolchain: ChromaDB, local embeddings (all-MiniLM-L6-v2), opsdesk_handbook.md mock data.)

Step-by-step

17. Read the mock handbook — note the POL-/PROC-/SLA-/SEC- source ids

```
cat data/opsdesk_handbook.md
```

18. Build the index: load → chunk on '## ' headers → embed → store

```
uv run python lab-04-knowledge-index/build_index.py
```

19. Run similarity search and study the top-3 sources and distances per question

```
uv run python lab-04-knowledge-index/similarity_search.py
```

20. Wrap the index as a FunctionTool and let an agent search it

```
uv run python lab-04-knowledge-index/index_agent.py
```

21. Change a question and predict the top-1 source before running — check yourself

Test it

build_index.py reports 10 stored chunks; the VPN question returns POL-001 and the P1 question returns PROC-101 as top-1, each with the smallest distance.

Note: The full scripts and a lab README are in labs/lab-04-knowledge-index/. Run every command from the labs/ folder with the uv run prefix.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 5 — Build and Evaluate a Grounded RAG Agent

Objective: LO2 — engineer grounding rules, then quantify retrieval efficiency.

Goal: Turn the index into a production-shaped RAG agent — mandatory retrieval, source citations, refusal when evidence is missing, prompt-injection defence — then measure hit@1, hit@3 and latency over a gold question set.

What you'll build

A grounded rag_agent with a retrieve() tool, plus an evaluation scoreboard for retrieval accuracy and latency. (Toolchain: AssistantAgent, FunctionTool retrieve(), reflect_on_tool_use, gold question set.)

Step-by-step

22. Read the four grounding rules in the rag_agent system message
23. Run the RAG agent on one answerable plus one unanswerable question

```
uv run python lab-05-rag-agent/rag_agent.py
```

24. Confirm the refusal: the annual-leave question must NOT be answered from thin air
25. Run the retrieval evaluation over the 6-question gold set

```
uv run python lab-05-rag-agent/evaluate_rag.py
```

26. Change n_results (top-k) and re-run — record how hit@3 and latency move

Test it

The VPN answer carries citations like (POL-001); the annual-leave question is refused; the scoreboard shows hit@1 ≥ 83% with millisecond-level latency.

Note: The full scripts and a lab README are in labs/lab-05-rag-agent/. Run every command from the labs/ folder with the uv run prefix.

Topic 03 — Implementing a Multi-Agent AI Workflow (28%)

ReAct · human-in-the-loop · teams · termination · SelectorGroupChat · feasibility

Key concepts

- ReAct (Reason + Act) is the loop behind every agent: Thought → Action (tool call) → Observation, repeated until the agent can answer.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Multi-agent frameworks — AutoGen, LangGraph, CrewAI — differ in orchestration style: conversation-driven teams, explicit state graphs, and role/task crews respectively.
- A RoundRobinGroupChat team shares one conversation and takes turns in a fixed order — the simplest reliable pattern for writer/reviewer workflows.
- Termination conditions (TextMentionTermination, MaxMessageTermination, combined with | and &) keep every team run bounded in messages, time and cost.
- Human-in-the-loop: a UserProxyAgent puts a person inside the team as an approval gate — essential where actions have real-world consequences.
- SelectorGroupChat uses an LLM selector that reads each agent's description to route the next turn dynamically — specialists get chosen, not scheduled.
- Feasibility of a multi-agent application is assessed on evidence: task fit, cost and latency per run, failure modes, guardrails, and readiness checks that pass.
- AutoGen Studio provides a no-code way to prototype the same teams, then export the configuration to code.

Lab 6 — Implement a ReAct Agent with Human-in-the-Loop

Objective: LO3 — expose the ReAct loop, then gate an agent behind human approval.

Goal: Hand-roll the Thought → Action → Observation loop so the ReAct pattern is visible, then build an AutoGen team where a UserProxyAgent makes YOU the approval gate before any draft is accepted.

What you'll build

A from-scratch ReAct agent over the ticket tools, and an approval workflow that stops only on APPROVE. (Toolchain: ReAct pattern, UserProxyAgent, input_func, TextMentionTermination, RoundRobinGroupChat.)

Step-by-step

27. Run the from-scratch ReAct loop and follow each Thought / Action / Observation

```
uv run python lab-06-react-human-in-loop/react_scratch.py
```

28. Identify in the transcript where the model plans vs where YOUR code acts
29. Run the human-in-the-loop team — you are the reviewer

```
uv run python lab-06-react-human-in-loop/human_in_loop.py
```

30. Request one revision ('make it more apologetic'), then approve with APPROVE
31. Note the safety net: TextMentionTermination('APPROVE') | MaxMessageTermination(12)

Test it

The ReAct agent reports the 15-minute P1 SLA target after two tool actions, and the approval team loops on your feedback and stops only when you type APPROVE.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: The full scripts and a lab README are in labs/lab-06-react-human-in-loop/. Run every command from the labs/ folder with the uv run prefix.

Lab 7 — Orchestrate a Round-Robin Multi-Agent Team

Objective: LO3 — compare manual orchestration with a RoundRobinGroupChat team.

Goal: Hand-roll a two-model writer/critic conversation to feel the orchestration cost, then run the same workflow as a RoundRobinGroupChat with proper termination conditions and a readable TaskResult.

What you'll build

A manual two-model conversation script, and a writer/reviewer team that stops on FINALISED. (Toolchain: RoundRobinGroupChat, TextMentionTermination, MaxMessageTermination, TaskResult.)

Step-by-step

32. Run the manual two-model conversation and count the plumbing it needs

```
uv run python lab-07-round-robin-team/two_model_conversation.py
```

33. Read the writer and reviewer system messages — the reviewer holds the checklist
34. Run the AutoGen round-robin team on the VPN announcement task

```
uv run python lab-07-round-robin-team/team_round_robin.py
```

35. Read the TaskResult: speaker sequence, message count and stop_reason
36. Remove MaxMessageTermination in your head: what could an unbounded team cost?

Test it

The team alternates writer → reviewer and stops on FINALISED (not the message cap), and you can read the stop reason straight from the TaskResult.

Note: The full scripts and a lab README are in labs/lab-07-round-robin-team/. Run every command from the labs/ folder with the uv run prefix.

Lab 8 — Route Specialists with SelectorGroupChat (Capstone)

Objective: LO3 — build the OpsDesk capstone and assess its feasibility with checks.

Goal: Assemble the capstone: a planner plus ticket and policy specialists routed dynamically by SelectorGroupChat, combining tools and RAG — then run the no-LLM readiness checks that decide whether the workflow is fit to ship.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

What you'll build

The complete OpsDesk multi-agent workflow, and a readiness report covering data, index health, retrieval accuracy and secrets hygiene. (Toolchain: SelectorGroupChat, agent descriptions, FunctionTool + RAG, readiness checks, AutoGen Studio (optional).)

Step-by-step

37. Read the three agent description fields — the selector routes by reading them
38. Run the capstone selector team on the TT-1002 VPN request

```
uv run python lab-08-selector-workflow-capstone/selector_team.py
```

39. Trace the route: planner → ticket_specialist → policy_specialist → final reply
40. Run the feasibility/readiness checks (no LLM needed)

```
uv run python lab-08-selector-workflow-capstone/run_checks.py
```

41. Fix any FAIL (e.g. rebuild the Lab 4 index) and re-run until clean
42. Optional: rebuild the team visually in AutoGen Studio

```
uv run --with autogenstudio autogenstudio ui
```

Test it

The team run ends on TERMINATE with a customer-ready reply citing POL-001, and run_checks.py prints 0 failed — your evidence-based feasibility verdict.

Note: The full scripts and a lab README are in labs/lab-08-selector-workflow-capstone/. Run every command from the labs/ folder with the uv run prefix.

Exam Preparation

- First pass: complete every lab in order — each builds on the artifacts before it.
- Second pass: redo the labs from a blank file until the AutoGen patterns (agent, tools, team, termination) are automatic.
- For LO1, be ready to name concrete LLM strengths and limitations you MEASURED (repeatability, latency, cost, privacy).
- For LO2, be ready to explain chunking, embeddings, similarity search, and how hit@k and latency quantify RAG efficiency.
- For LO3, be ready to justify a single-agent vs multi-agent design and cite the feasibility evidence from Lab 8's checks.
- Sharpen exam technique with the free Tertiary Infotech practice exams: https://exams.tertiaryinfotech.com/practice-exams
- The Practical Performance mirrors the labs — keep the Learner Guide and your working lab folder open (open book).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Glossary

- **LLM (Large Language Model) —** A model trained on large text corpora that generates language token by token (e.g. GPT-4o, Claude, Gemini, Llama).
- **AI agent —** An LLM wrapped with a role (system message), conversation context and tools so it can act, not just answer.
- **AutoGen —** Microsoft's open-source multi-agent framework: autogen-core, autogen-agentchat, autogen-ext and AutoGen Studio.
- **AssistantAgent —** AgentChat's standard LLM agent: model client + system message + optional tools and structured output.
- **Model client —** The provider abstraction (e.g. OpenAIChatCompletionClient) that lets the same agent run on OpenAI, Azure, Gemini or Ollama.
- **FunctionTool —** An AutoGen wrapper that turns a typed Python function into a callable tool with an auto-generated schema.
- **Function calling —** The mechanism where the model requests a tool invocation and YOUR code executes it and returns the result.
- **Structured output —** Forcing the model to reply in a validated Pydantic schema (output_content_type) instead of free text.
- **RAG (Retrieval-Augmented Generation) —** Retrieving relevant chunks from your own data and adding them to the prompt so answers are grounded and citable.
- **Embedding —** A vector representation of text; semantically similar texts have nearby vectors (cosine similarity).
- **Chunking —** Splitting documents into retrievable units; boundaries directly affect retrieval quality.
- **Vector database —** A store (e.g. ChromaDB) that indexes embeddings for fast approximate nearest-neighbour search.
- **hit@k —** A retrieval metric: the fraction of gold questions whose expected source appears in the top-k results.
- **ReAct —** The Reason + Act pattern: Thought → Action (tool call) → Observation, looped until the agent can answer.
- **RoundRobinGroupChat —** An AgentChat team where participants take turns in a fixed order in one shared conversation.
- **SelectorGroupChat —** An AgentChat team where a selector LLM reads agent descriptions and chooses the next speaker dynamically.
- **Termination condition —** A rule (TextMentionTermination, MaxMessageTermination, combined with | / &) that ends a team run.
- **UserProxyAgent —** An agent that routes its turn to a human (input()), putting a person in the loop as an approval gate.
- **Human-in-the-loop —** A workflow design where a human must review or approve agent output before it takes effect.
- **Prompt injection —** Malicious instructions hidden in retrieved content; agents must treat retrieved text as data, never as commands.
- **uv —** A fast Python package and project manager: uv init, uv add, uv sync, uv run.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **AutoGen Studio —** A local web UI for building and testing agent teams without code.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg